



Prévision des Accidents de Ski Liés aux Avalanches

Rapport de Stage

NDIAYE Souleymane
Encadré par M. Olivier Wintenberger

Master 1 ISDS

Laboratoire LPSM, Sorbonne Université

Table des matières

1	Introduction	3
2	Données Utilisées	3
2.1	Source des données	3
2.2	Description du jeu de données	3
3	Méthodologie	4
3.1	Projection des points sur un tracé	4
3.2	Définition mathématique	4
3.3	Projection orthogonale d'un point sur un tracé	4
3.4	Module Python pour la projection orthogonale	4
3.4.1	Description des fonctions	4
3.4.2	Fonction <code>latlon_to_cartesian</code>	4
3.4.3	Fonction <code>projection_orthogonale</code>	5
3.4.4	Fonction <code>projection_orthogonale_sur_trajet</code>	5
3.4.5	Objectif global du module	5
3.4.6	Figure de la projection orthogonale	6
3.5	Module Python identification du tracé dans un espace clos	6
3.5.1	Description du Module Python identifier_tracer	6
3.5.2	Rôle global	6
3.5.3	Description des fonctions	6
3.5.4	Fonction <code>extraire_traces(dataframe)</code>	6
3.5.5	Fonction <code>norme2(point1, point2)</code>	7
3.5.6	Fonction <code>identifier_trace(r, lat_accident, lon_accident, dataframe)</code>	7
3.5.7	Objectif du module	7
3.5.8	Figure d'un scénario d'accident de ski	8
4	Définition du Modèle GLM	8
4.1	Implémentation du modèle GLM	9
4.2	Résultats de la régression logistique	9
4.3	Analyse de la courbe ROC	10
4.3.1	Définition et rôle de la courbe ROC	10
4.3.2	Analyse de la courbe ROC	10
4.3.3	Interprétation de la courbe	11
5	Amélioration de Modèle	11
5.1	Encodage one-hot pour les orientations	11
5.2	Analyse du modèle GLM amélioré avec les orientations	11
5.2.1	Introduction de nouvelles variables explicatives	11
5.2.2	Rapport de classification	11
5.2.3	Matrice de confusion	12
5.2.4	Score AUC de la courbe ROC	12
5.2.5	Interprétation des résultats	12
5.3	Courbe ROC du modèle GLM amélioré avec orientations	12
5.3.1	Analyse de la courbe ROC	13
6	Conclusion	13
7	Annexes	14

21 septembre 2026

Remerciements

Je tiens à remercier M. Olivier Wintenberger pour son encadrement, sa patience, et son soutien tout au long de ce projet. Son expertise et ses conseils avisés ont été d'une grande aide, et m'ont permis de progresser tant sur le plan technique que professionnel. Je remercie également M. Joseph Vilmarest pour son soutien tout au long de se stage, ainsi que l'équipe du LPSM pour leur accueil chaleureux.

1 Introduction

Les avalanches représentent un danger majeur pour les skieurs, en particulier dans le contexte actuel de changements climatiques. Ces changements influencent non seulement la fréquence et l'intensité des avalanches, mais également les facteurs déclencheurs associés. Ce stage s'inscrit dans un projet visant à mieux comprendre et prédire les accidents de ski liés aux avalanches à partir de données réelles et de modèles statistiques avancés.

L'objectif de ce stage était double :

- Collecter et analyser des données sur les avalanches et les accidents associés.
- Implémenter un modèle GLM (Generalized Linear Model) pour prédire les accidents en fonction de facteurs tels que le risque et l'altitude, puis améliorer ce modèle en y intégrant des variables supplémentaires et en effectuant une visualisation des résultats.

2 Données Utilisées

Dans cette analyse, nous avons pris en compte les **sommets** de l'API Skitour comme référence .

- Pour chaque accident recensé dans la base ANENA (période allant de 2004 à 2023), nous avons identifié le **sommet le plus proche** du point d'accident.
- Cette approche permet de mieux contextualiser l'analyse en se basant sur des zones géographiques précises et des sommets spécifiques fréquentés par les skieurs et alpinistes.

Le choix des sommets permet de capturer des informations plus cohérentes sur les facteurs géographiques et environnementaux, notamment les caractéristiques des sommets situés dans un voisinage restreint autour du point d'accident. Les résultats de cette approche sont présentés dans la section suivante.

2.1 Source des données

Les données proviennent de trois principales sources :

1. Les archives des accidents d'avalanches disponibles sur le site de l'ANENA pour la période 2004-2023.
2. Les sommets enregistrées sur l'API Skitour.
3. les données historiques des avalanches sur les massifs en interrogeant le site data-gouv.fr pour obtenir les niveaux de risque associés.

Pour chaque observation, une valeur binaire a été attribuée à la colonne **accident** :

- 1 pour les accidents recensés dans les archives ANENA.
- 0 pour les sommets sans accident issues de Skitour.

2.2 Description du jeu de données

Le jeu de données final contient 11265 observations, réparties comme suit :

- 519 observations indiquant un accident (**accident** = 1).
- 11265 observations sans accident (**accident** = 0).

Les colonnes incluent les variables suivantes :

- **Altitude** : Altitude en mètres.
- **Risque** : Indice de risque d'avalanche (1 à 5).
- **Orientation** : Orientation géographique (par exemple, Sud, Sud-Est).

— **Latitude** et **Longitude** : Coordonnées géographiques.

3 Méthodologie

3.1 Projection des points sur un tracé

Dans un premier temps, nous avons étudié comment projeter un point (par exemple, l'emplacement d'un accident) sur un tracé GPX dans un espace défini par un rayon r autour du point. La méthode consiste à identifier le tracé le plus proche en minimisant la distance euclidienne (norme L_2).

Cette projection permet de relier des points accidentés à des caractéristiques naturelles telles que :

- La météo (température, vent, humidité).
- Les caractéristiques du terrain (altitude, Risque, orientation).

3.2 Définition mathématique

Soit un espace vectoriel euclidien R^n muni de la **norme 2** (ou norme euclidienne), notée $\|\cdot\|$, et soit L un sous-espace affine de R^n (par exemple une droite ou un plan). Si on considère un point $P \in R^n$, la **projection orthogonale** de P sur L est le point P_L appartenant à L tel que la distance euclidienne entre P et P_L soit minimale.

Formellement, la **projection orthogonale** de P sur L est définie par :

$$P_L = \underset{Q \in L}{\operatorname{argmin}} \|P - Q\|_2$$

Autrement dit, P_L est le point $Q \in L$ qui minimise la norme 2 (ou distance euclidienne) entre P et L .

3.3 Projection orthogonale d'un point sur un tracé

Pour résoudre ce problème, nous avons commencé par développer une fonction qui projette un point défini par sa latitude et sa longitude sur un tracé, représenté comme une série de segments de ligne. Nous avons utilisé une méthode de projection orthogonale, où nous projetons le point sur le segment le plus proche du tracé. La **projection orthogonale** d'un point sur un sous-espace est un concept clé en géométrie euclidienne, et la norme 2 (ou norme euclidienne) est la norme classique utilisée pour mesurer la distance dans cet espace.

3.4 Module Python pour la projection orthogonale

Ce module Python contient des fonctions qui permettent de travailler avec des trajectoires géographiques définies par des paires de latitude et de longitude. Son objectif principal est de calculer la **projection orthogonale** d'un point donné sur une trajectoire composée de plusieurs segments (chaque segment étant défini par deux points successifs de la trajectoire).

3.4.1 Description des fonctions

3.4.2 Fonction `latlon_to_cartesian`

- **Entrées** :

- `lat` : latitude (en degrés) d'un point.
- `lon` : longitude (en degrés) d'un point.
- **Sortie** :
 - Un vecteur $[x, y]$ représentant les coordonnées cartésiennes approximatives du point.
- **Rôle** : Convertit des coordonnées géographiques (latitude, longitude) en coordonnées cartésiennes (x, y) sur une projection 2D de la surface de la Terre, avec un rayon moyen terrestre ($R = 6371000$ m). Cela simplifie les calculs en évitant de travailler directement avec les coordonnées sphériques.

3.4.3 Fonction `projection_orthogonale`

- **Entrées** :
 - `point` : un vecteur $[x, y]$, représentant les coordonnées cartésiennes du point à projeter.
 - `segment_start` : un vecteur $[x, y]$, représentant l'origine du segment.
 - `segment_end` : un vecteur $[x, y]$, représentant l'extrémité du segment.
- **Sortie** :
 - Les coordonnées cartésiennes $[x, y]$ du point projeté orthogonalement sur le segment donné.
- **Rôle** : Cette fonction calcule la projection orthogonale d'un point donné sur un segment défini par `segment_start` et `segment_end`. Si la projection dépasse les extrémités du segment, elle est ramenée aux limites du segment (i.e. $t \in [0, 1]$).

3.4.4 Fonction `projection_orthogonale_sur_trajet`

- **Entrées** :
 - `lat` : latitude du point à projeter.
 - `lon` : longitude du point à projeter.
 - `trajet` : une liste de tuples (lat, lon) représentant les points successifs formant une trajectoire géographique.
- **Sortie** :
 - Les coordonnées géographiques (latitude, longitude) du point projeté orthogonalement sur le trajet.
- **Rôle** :
 - Convertir le point donné en coordonnées cartésiennes.
 - Parcourir chaque segment du trajet pour calculer la projection orthogonale du point sur ce segment.
 - Identifier le point projeté le plus proche parmi toutes les projections obtenues.
 - Convertir ce point projeté (en coordonnées cartésiennes) en latitude/longitude avant de le retourner.

3.4.5 Objectif global du module

L'objectif du module est de calculer **le point le plus proche sur une trajectoire géographique** à partir d'un point donné. Ce calcul est particulièrement utile dans des contextes comme :

- Le suivi des itinéraires (par exemple, déterminer la position exacte d'un utilisateur par rapport à une route prédéfinie).
- L'analyse des déplacements géographiques.
- Le calcul de distances minimales par rapport à des trajets définis.

3.4.6 Figure de la projection orthogonale

Voici une représentation géométrique de la projection d'un point sur une trajectoire. Le point extérieur est projeté orthogonalement sur le segment le plus proche du tracé, indiqué par une ligne pointillée perpendiculaire. Cela illustre visuellement comment la projection se fait par rapport au tracé.

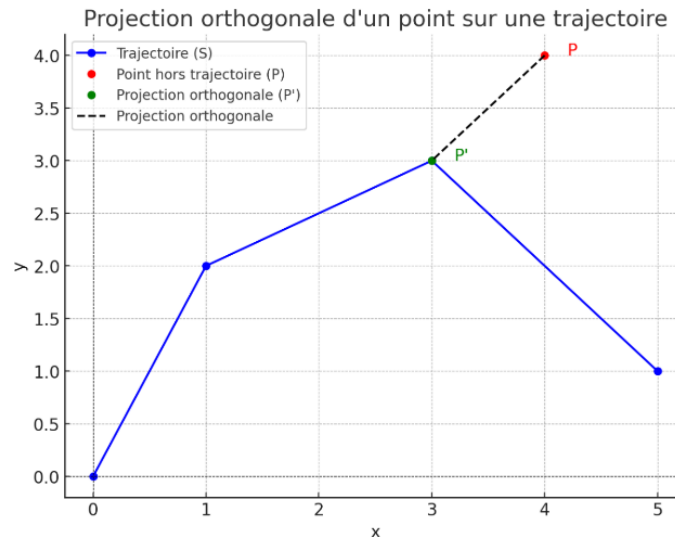


FIGURE 1 – Projection orthogonale d'un point sur un tracé.

3.5 Module Python identification du tracé dans un espace clos

Dans le cadre de cette étude, nous avons développé une fonction permettant de parcourir un ensemble de tracés dans un espace défini par un rayon r autour du point de l'accident. L'objectif est de déterminer le tracé le plus proche en utilisant la norme 2.

3.5.1 Description du Module Python identifier_tracer

Ce module Python permet d'identifier le trajet le plus proche d'un point d'accident donné parmi plusieurs trajets géographiques. Il s'appuie sur la distance orthogonale pour déterminer ce trajet et assure que cette distance est inférieure ou égale à un rayon donné r .

3.5.2 Rôle global

Le module identifie l'identifiant (`id`) d'un trajet à partir de données géographiques et d'un point d'accident en se basant sur la distance orthogonale minimale entre le point d'accident et les trajets.

3.5.3 Description des fonctions

3.5.4 Fonction `extraire_traces(dataframe)`

- **Rôle :** Transforme un DataFrame en un dictionnaire contenant des trajets, où chaque clé correspond à un identifiant unique `id`, et chaque valeur est une liste de coordonnées géographiques (latitude, longitude).
- **Entrée :**

- **dataframe** : Un DataFrame contenant au moins les colonnes `latitude`, `longitude` et `id`.
- **Sortie** : Un dictionnaire sous la forme :

$$\text{traces} = \{\text{id}_1 : [(\text{lat}_1, \text{lon}_1), (\text{lat}_2, \text{lon}_2), \dots], \text{id}_2 : [(\text{lat}_1, \text{lon}_1), (\text{lat}_2, \text{lon}_2), \dots], \dots\}$$

3.5.5 Fonction `norme2(point1, point2)`

- **Rôle** : Calcule la distance euclidienne (norme 2) entre deux points en 2D.
- **Entrées** :
 - `point1` : Coordonnées du premier point (x_1, y_1) .
 - `point2` : Coordonnées du second point (x_2, y_2) .
- **Sortie** : Un scalaire représentant la distance euclidienne entre les deux points :

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3.5.6 Fonction `identifier_trace(r, lat_accident, lon_accident, dataframe)`

- **Rôle** : Identifie l'identifiant *id* du trajet le plus proche d'un point d'accident donné, et qui respecte la contrainte $\text{distance} \leq r$.
- **Étapes principales** :
 1. Appelle la fonction `extraire_traces` pour convertir le DataFrame en un dictionnaire de trajets.
 2. Parcourt tous les trajets du dictionnaire.
 3. Pour chaque trajet :
 - Calcule la projection orthogonale du point d'accident sur le trajet en utilisant la fonction `projection_orthogonale_sur_trajet`.
 - Calcule la distance entre le point d'accident et cette projection à l'aide de la fonction `norme2`.
 4. Met à jour l'identifiant *id* du trajet et la distance minimale si une distance plus petite est trouvée et qu'elle est inférieure ou égale au rayon *r*.
- **Entrées** :
 - *r* : Rayon en mètres, définissant une limite pour les trajets considérés.
 - `lat_accident` : Latitude du point d'accident.
 - `lon_accident` : Longitude du point d'accident.
 - `dataframe` : Un DataFrame contenant les colonnes `latitude`, `longitude` et `id`.
- **Sorties** :
 - `id_trajet` : L'identifiant du trajet le plus proche respectant $\text{distance} \leq r$.
 - `projection` : Les coordonnées géographiques (`latitude`, `longitude`) du point projeté sur le trajet identifié.

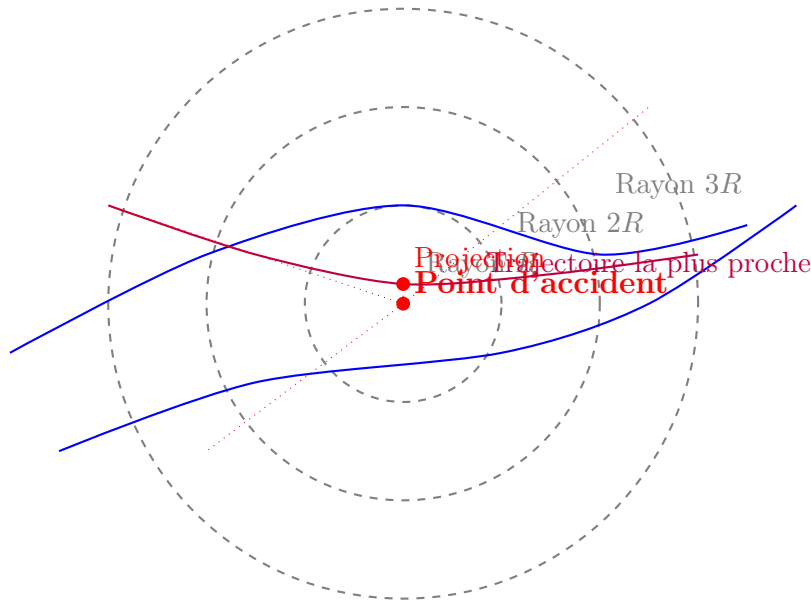
3.5.7 Objectif du module

Ce module est conçu pour des analyses géographiques, par exemple :

- Identifier le trajet le plus proche d'un point d'accident dans un espace défini.
- Fournir une base pour analyser les facteurs (météorologiques, obstacles, etc.) ayant pu causer un accident en se basant sur la position d'un skieur par rapport aux trajets.

3.5.8 Figure d'un scénario d'accident de ski

Voici un schéma géométrique illustrant le scénario d'un accident de ski. Il montre les différents tracés (lignes de trajectoire), le point de l'accident représenté en rouge, et la zone circulaire de rayon r autour du point d'accident. La projection du point d'accident sur le tracé le plus proche est marquée avec une ligne pointillée perpendiculaire, et le tracé le plus proche est mis en évidence. Ce diagramme vous aide à visualiser comment le tracé le plus proche est identifié en fonction du rayon r .



4 Définition du Modèle GLM

Un modèle linéaire généralisé (GLM) est une généralisation des modèles linéaires classiques qui permet de modéliser une variable réponse Y suivant une distribution de la famille exponentielle (par exemple, la loi normale, binomiale ou Poisson). Il repose sur trois composantes principales :

- **La fonction de lien** : Une fonction monotone g qui relie la moyenne de la variable réponse $E[Y] = \mu$ à une combinaison linéaire des variables explicatives $\mathbf{X}$, telle que :

$$g(\mu) = \mathbf{X}^\top \boldsymbol{\beta},$$

où $\boldsymbol{\beta}$ représente les coefficients du modèle.

- **La distribution de la variable réponse** : La variable Y suit une distribution appartenant à la famille exponentielle, par exemple :

$$f_Y(y; \theta, \phi) = \exp \left(\frac{y\theta - b(\theta)}{\phi} + c(y, \phi) \right),$$

où θ est le paramètre canonique, ϕ un paramètre de dispersion, et $b(\cdot)$ et $c(\cdot)$ sont des fonctions spécifiques à la distribution.

- **La prédiction linéaire** : Le prédicteur linéaire est exprimé comme :

$$\eta = \mathbf{X}^\top \boldsymbol{\beta}.$$

L'estimation des paramètres $\boldsymbol{\beta}$ est réalisée par maximisation de la vraisemblance (ou équivalent, minimisation de la déviance). Cela se fait typiquement via des algorithmes numériques tels que l'algorithme itératif de repondération des moindres carrés (IRLS).

Dans notre implémentation, nous avons utilisé `statsmodels` et `scikit-learn` en Python] pour ajuster un modèle GLM avec une fonction de lien spécifique et une distribution adaptée à notre problème.

4.1 Implémentation du modèle GLM

Nous avons utilisé un modèle GLM pour prédire la variable `accident` (binaire) en fonction de plusieurs variables explicatives :

- **Altitude** : Variable continue.
- **Risque** : Variable continue.

Le modèle est défini comme suit :

$$\text{logit}(P(\text{accident} = 1)) = \beta_0 + \beta_1 \cdot \text{Risque} + \beta_2 \cdot \text{Altitude}$$

- $(\beta_0, \beta_1, \beta_2)$: Coefficients du modèle, à estimer.

4.2 Résultats de la régression logistique

Nous avons ajusté un modèle de régression logistique pour prédire la probabilité d'accidents liés aux avalanches en fonction de l'altitude et du risque. Les résultats sont présentés dans le tableau ci-dessous :

TABLE 1 – Résumé des résultats de la régression logistique

Variable	Coefficient (β)	Erreur standard	Valeur-p
Constante (const)	-16.4558	0.552	< 0.001
Altitude	0.0030	0.000	< 0.001
Risque	3.1166	0.097	< 0.001

Interprétation des coefficients :

- *Constante* : La constante indique que, lorsque les variables explicatives (altitude et risque) sont à zéro, la probabilité d'accident est extrêmement faible. Cela correspond à une situation théorique et n'a pas de signification pratique directe.
- *Altitude* : Une augmentation de 1 mètre d'altitude est associée à une augmentation marginale mais significative de la probabilité d'accident, avec un coefficient de 0.003.
- *Risque* : Le risque d'avalanche (noté sur une échelle de 1 à 5) a un effet fortement positif et significatif sur la probabilité d'accident, avec un coefficient de 3.1166.

La qualité globale du modèle, mesurée par la pseudo R^2 , est de ****0.4838****, ce qui indique que près de 48.38% de la variance de la variable cible (accident) est expliquée par les variables explicatives. Le log-vraisemblance final est de ****1098.5****, avec un test du rapport de vraisemblance donnant une valeur-p < 0.001, suggérant que le modèle est globalement significatif.

Conclusion : Les résultats obtenus à l'aide d'une base de données étendue confirment l'importance des variables explicatives altitude et risque dans la prédiction des accidents liés aux avalanches. L'amélioration de la qualité du modèle (Pseudo $R^2 = 0.4838$) reflète une meilleure capacité à expliquer la variance dans les données. Toutefois, le rappel relativement faible pour la classe "Accident" souligne la nécessité de poursuivre les travaux pour améliorer la détection des accidents, notamment en explorant d'autres variables explicatives ou en équilibrant les données.

Les premières analyses montrent une corrélation significative entre le risque d’avalanche et la probabilité d’accident. L’altitude joue également un rôle important, les accidents ayant tendance à se produire à des altitudes plus élevées.

4.3 Analyse de la courbe ROC

4.3.1 Définition et rôle de la courbe ROC

La courbe ROC (*Receiver Operating Characteristic*) est un outil graphique utilisé pour évaluer les performances d’un modèle de classification binaire. Elle représente la relation entre le taux de vrais positifs (TPR, *True Positive Rate*) et le taux de faux positifs (FPR, *False Positive Rate*) à différents seuils de décision.

- Le TPR, aussi appelé sensibilité, mesure la proportion de vrais positifs correctement identifiés parmi tous les cas positifs.
- Le FPR mesure la proportion de faux positifs parmi tous les cas négatifs.

Le rôle principal de la courbe ROC est de comparer les performances d’un modèle à un seuil de classification aléatoire (la diagonale en pointillés dans le graphique). Plus la courbe ROC est proche du coin supérieur gauche, plus le modèle est performant. La qualité globale de la classification est résumée par l’AUC (*Area Under Curve*), qui correspond à l’aire sous la courbe ROC. Une valeur de 0.5 indique une performance équivalente à une classification aléatoire, tandis qu’une valeur proche de 1 représente une très bonne performance.

4.3.2 Analyse de la courbe ROC

La courbe ROC (*Receiver Operating Characteristic*) permet d’évaluer les performances du modèle de régression logistique utilisé pour prédire les accidents. Elle illustre la relation entre le **taux de vrais positifs** (TPR, également appelé sensibilité) et le **taux de faux positifs** (FPR).

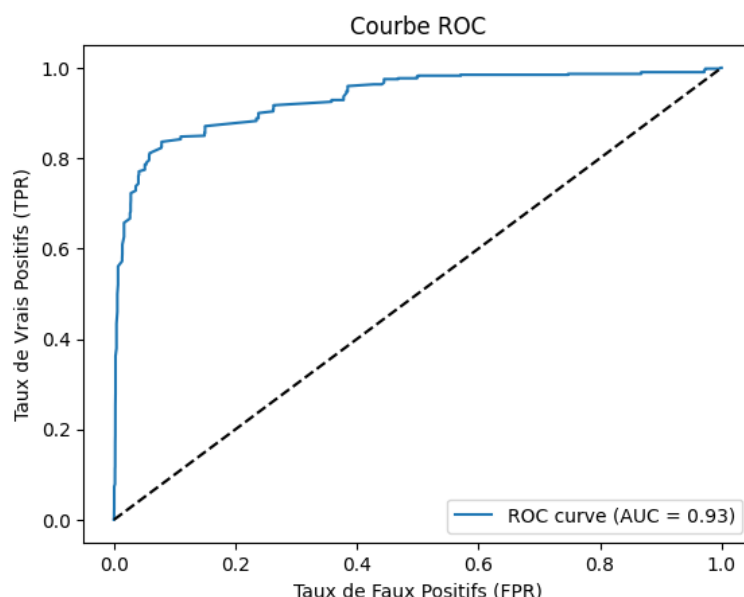


FIGURE 2 – Courbe ROC du modèle de régression logistique

4.3.3 Interprétation de la courbe

- **Aire sous la courbe (AUC)** : La valeur de l'AUC est de 0.93, ce qui indique une excellente capacité prédictive du modèle. En effet, une AUC proche de 1 signifie que le modèle distingue efficacement les classes positives (accidents) des classes négatives (non-accidents). Plus précisément, une AUC de 0.93 indique que dans 93% des cas, le modèle attribue une probabilité plus élevée aux observations positives qu'aux observations négatives.
- **Forme de la courbe** : La courbe ROC est proche de l'axe des ordonnées dans sa partie gauche, ce qui démontre que le modèle parvient à capturer une majorité de vrais positifs tout en maintenant un faible taux de faux positifs. Cela témoigne de la robustesse du modèle.
- **Seuils de décision** : La courbe ROC permet d'identifier différents seuils de décision selon les besoins :
 - *Faible seuil* : Plus de vrais positifs, mais également plus de faux positifs.
 - *Seuil élevé* : Moins de faux positifs, mais au risque de manquer des accidents.

Conclusion : Le modèle présente de **très bonnes performances** pour discriminer les accidents et les non-accidents. Avec une AUC de 0.93, il est fiable et robuste pour des analyses prédictives. L'utilisation de cette courbe permet également d'ajuster le seuil de décision en fonction du contexte opérationnel, par exemple pour réduire les faux positifs ou maximiser la détection des accidents.

5 Amélioration de Modèle

5.1 Encodage one-hot pour les orientations

Pour intégrer les orientations (variables catégoriques) dans le modèle, nous avons utilisé un encodage one-hot. Chaque orientation géographique a été représentée par une colonne binaire :

- **Orientation_Sud** : 1 si l'orientation est Sud, 0 sinon.
- **Orientation_Sud-Est** : 1 si l'orientation est Sud-Est, 0 sinon.
- Et ainsi de suite pour les autres orientations.

Le modèle final devient :

$$\text{logit}(P(\text{accident} = 1)) = \beta_0 + \beta_1 \cdot \text{Risque} + \beta_2 \cdot \text{Altitude} + \sum_i \beta_{i+2} \cdot \text{Orientation}_i$$

- $(\beta_0, \beta_1, \beta_2, \beta_{i+2})$: Coefficients du modèle, à estimer.

5.2 Analyse du modèle GLM amélioré avec les orientations

5.2.1 Introduction de nouvelles variables explicatives

Pour améliorer la précision des prédictions, les orientations ont été ajoutées comme nouvelles variables explicatives dans le modèle GLM en utilisant l'encodage *one-hot*. Cela a permis de représenter chaque orientation (par exemple, Sud, Sud-Est, etc.) par des colonnes binaires. Cette nouvelle version du modèle a été évaluée à l'aide de la bibliothèque Scikit-learn.

5.2.2 Rapport de classification

Le rapport de classification est présenté dans le tableau 2.

TABLE 2 – Rapport de classification après amélioration du modèle

Classe	Précision	Recall	F1-Score	Support
0 (Non-Accident)	0.98	1.00	0.99	2242
1 (Accident)	0.92	0.57	0.71	115
Accuracy		0.98		
Macro Moyenne	0.95	0.79	0.85	2357
Moyenne pondérée	0.98	0.98	0.97	2357

5.2.3 Matrice de confusion

La matrice de confusion, qui permet de visualiser les erreurs de classification, est représentée dans le tableau 3.

TABLE 3 – Matrice de confusion après amélioration du modèle

	Prédit : 0	Prédit : 1
Réel : 0	2236	6
Réel : 1	49	66

5.2.4 Score AUC de la courbe ROC

La performance globale du modèle est également évaluée via l'**aire sous la courbe ROC (AUC)**. Après amélioration, le score AUC atteint une valeur de **0.97**, démontrant la capacité élevée du modèle à discriminer les classes (accidents et non-accidents).

$$\text{AUC} = 0.9729 \quad (1)$$

5.2.5 Interprétation des résultats

- **Classe 0 (Non-Accident)** : Le modèle atteint une très haute précision (0.98) et un rappel de 1.00, indiquant qu'il identifie correctement presque tous les non-accidents.
- **Classe 1 (Accident)** : Bien que la précision reste élevée (0.92), le rappel est de 0.57. Cela signifie que 43% des accidents ne sont pas détectés.
- **Score global** : L'accuracy atteint 0.98 et l'AUC de 0.97 confirme l'excellente performance générale du modèle.

Conclusion : L'intégration des orientations comme variables explicatives a significativement amélioré les performances globales du modèle, comme le montre l'augmentation de l'AUC (0.97) et les scores élevés de précision et de rappel. Cependant, des ajustements sont nécessaires pour mieux capturer la classe minoritaire des accidents.

5.3 Courbe ROC du modèle GLM amélioré avec orientations

La courbe ROC (Receiver Operating Characteristic) permet d'évaluer les performances globales du modèle en représentant le compromis entre le taux de vrais positifs (TPR) et le taux de faux positifs (FPR). La Figure 3 présente la courbe ROC obtenue après amélioration du modèle.

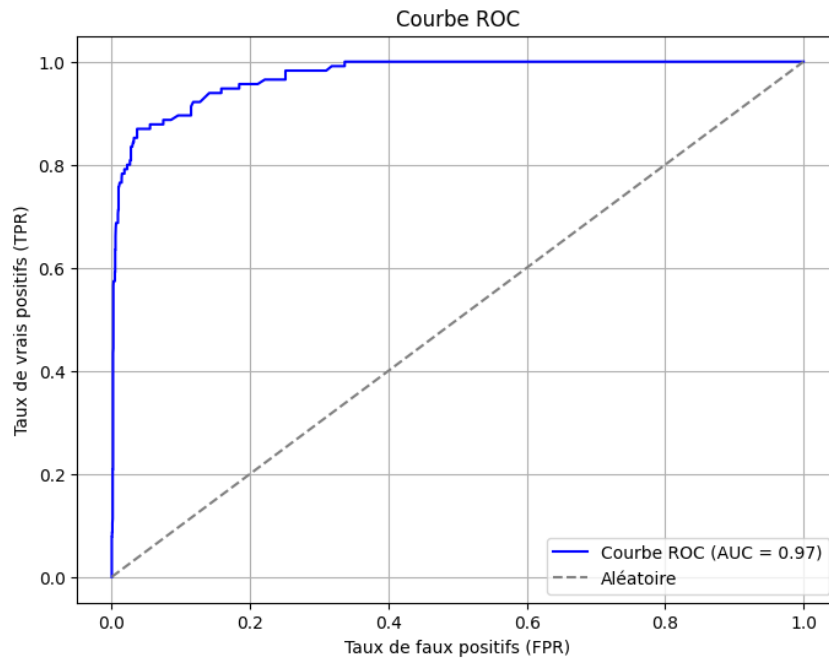


FIGURE 3 – Courbe ROC du modèle amélioré avec $AUC = 0.97$

5.3.1 Analyse de la courbe ROC

- **AUC (Area Under the Curve)** : La valeur de l'aire sous la courbe est de **0.97**, ce qui indique une excellente capacité de discrimination du modèle.
- **Taux de vrais positifs (TPR)** : Le modèle atteint un taux élevé de détection des accidents (classe positive) tout en minimisant le taux de faux positifs (FPR).
- **Comparaison à la ligne aléatoire** : La ligne pointillée représente un modèle aléatoire, pour lequel l'AUC serait de 0.5. L'écart significatif entre la courbe ROC et cette ligne montre que notre modèle est nettement performant.

Conclusion La courbe ROC et l'AUC montrent que l'intégration des orientations a significativement amélioré la performance du modèle. Ces résultats confirment la pertinence de considérer les orientations comme une variable explicative dans le cadre de la prévision des accidents liés aux avalanches.

6 Conclusion

Ce stage a permis d'implémenter un modèle GLM pour prédire les accidents de ski liés aux avalanches en fonction de facteurs environnementaux. Les résultats montrent que le risque et l'altitude sont des variables explicatives significatives.

Les travaux futurs incluent :

- L'intégration de données climatiques supplémentaires (par exemple, température, vent).
- L'utilisation de modèles plus complexes tels que les modèles à états cachés pour capturer des dynamiques temporelles.
- Une visualisation 3D des prédictions pour faciliter l'interprétation des résultats.

Références

- [1] ANENA. Archives et données d'accidents d'avalanches en France. <https://anena.org/accidents/archives-et-donnees-daccidents-davalanche-en-france>.
- [2] Skitour. API Skitour. <https://skitour.fr/api>.
- [3] de Vilmares, J. (2022). *Modèles espace-état pour la prévision de séries temporelles. Application aux marchés électriques*. (Doctoral dissertation, Sorbonne université).
- [4] Eckert, N., Corona, C., Giacona, F., Gaume, J., Mayer, S., van Herwijnen, A., ... Stoffel, M. (2024). Climate change impacts on snow avalanche activity and related risks. *Nature Reviews Earth Environment*, 1-21.
- [5] Degraeuwe, B., Schmudlach, G., Winkler, K., Köhler, J. (2024). SLABS : An improved probabilistic method to assess the avalanche risk on backcountry ski tours. *Cold Regions Science and Technology*, 221, 104169.
- [6] Eschner, J. (2023). Real-Time Avalanche Risk Visualization on a Large-Scale Geospatial Dataset, *Diploma Thesis*, TU Wien.

7 Annexes

Implémentation en Python

Voici la fonction Python qui réalise cette projection orthogonale d'un point sur un tracé.

```
import numpy as np

# Fonction pour convertir latitude/longitude en coordonnées cartésiennes approximatives
def latlon_to_cartesian(lat, lon):
    R = 6371000 # Rayon moyen de la Terre en mètres
    x = R * np.cos(np.radians(lat)) * np.cos(np.radians(lon))
    y = R * np.cos(np.radians(lat)) * np.sin(np.radians(lon))
    return np.array([x, y])

# Fonction pour calculer la projection orthogonale d'un point sur un segment de ligne
def projection_orthogonale(point, segment_start, segment_end):
    # Calculer le vecteur du segment
    segment_vec = segment_end - segment_start
    point_vec = point - segment_start

    # Projeter le vecteur point sur le vecteur du segment
    segment_length_squared = np.dot(segment_vec, segment_vec)
    if segment_length_squared == 0: # Segment de longueur nulle
        return segment_start # Retourner l'origine du segment

    t = np.dot(point_vec, segment_vec) / segment_length_squared
    t = max(0, min(1, t)) # Limiter t pour qu'il reste sur le segment [0, 1]

    projection = segment_start + t * segment_vec
    return projection
```

```

# Fonction principale
def projection_orthogonale_sur_trajet(lat, lon, trajet):
    # Convertir le point (lat, lon) en coordonnées cartésiennes
    point = latlon_to_cartesian(lat, lon)

    # Initialisation de la distance minimale et de la meilleure projection
    meilleure_projection = None
    distance_minimale = float('inf')

    # Parcourir chaque segment du trajet
    for i in range(len(trajet) - 1):
        segment_start = latlon_to_cartesian(trajet[i][0], trajet[i][1])
        segment_end = latlon_to_cartesian(trajet[i+1][0], trajet[i+1][1])

        # Calculer la projection orthogonale sur ce segment
        projection = projection_orthogonale(point, segment_start, segment_end)

        # Calculer la distance entre le point et la projection
        distance = np.linalg.norm(projection - point)

        # Si cette projection est plus proche que les précédentes, on la garde
        if distance < distance_minimale:
            distance_minimale = distance
            meilleure_projection = projection

    # Convertir la meilleure projection en latitude/longitude (inverse de la conversion)
    def cartesian_to_latlon(cartesian):
        R = 6371000
        lon = np.degrees(np.arctan2(cartesian[1], cartesian[0]))
        lat = np.degrees(np.arctan2(cartesian[0], np.sqrt(cartesian[0]**2 + cartesian[1]**2)))
        return lat, lon

    return cartesian_to_latlon(meilleure_projection)

```

Cette fonction prend en entrée un couple de latitude et longitude, ainsi qu'un trajet composé de plusieurs segments, et retourne la projection orthogonale du point sur le tracé.

Algorithme Python pour l'identification du tracé

Voici l'algorithme en Python utilisé pour réaliser cette tâche :

```

import numpy as np

def extraire_traces(dataframe):
    """
    Transforme un DataFrame en dictionnaire de trajets groupés par identifiant.

    Args:
        dataframe (pd.DataFrame): Le DataFrame contenant les colonnes 'latitude', 'longitude' et 'identifiant'.
    """

```

```

Returns:
    dict: Un dictionnaire où les clés sont les identifiants (id) et les valeurs
          sont des listes de tuples (latitude, longitude).
"""
traces = {}
for identifiant, group in dataframe.groupby('id'):
    traces[identifiant] = list(zip(group['latitude'], group['longitude']))
return traces

# Fonction pour calculer la norme 2 entre deux points
def norme2(point1, point2):
    return np.linalg.norm(np.array(point1) - np.array(point2))

# Fonction principale
def identifier_trace(r, lat_accident, lon_accident, dataframe):

    # Convertir le DataFrame en dictionnaire de trajets
    traces = extraire_traces(dataframe)

    # Initialiser les variables pour stocker l'identité du tracé et la distance minimale
    id_trajet = None
    distance_minimale = float('inf')

    # Parcourir tous les tracés dans l'espace
    for identifiant, trajet in traces.items():
        # Calculer la projection orthogonale du point de l'accident sur le tracé
        projection = projection_orthogonale_sur_trajet(lat_accident, lon_accident, trajet)

        # Calculer la distance entre le point de l'accident et la projection
        distance = norme2((lat_accident, lon_accident), projection)

        # Vérifier si cette distance est inférieure au rayon et à la distance minimale
        if distance <= r and distance < distance_minimale:
            distance_minimale = distance
            id_trajet = identifiant

    return id_trajet, projection

```

Cette fonction prend en paramètre le rayon r , les coordonnées du point de l'accident et un dictionnaire de tracés, et retourne l'identifiant du tracé le plus proche de l'accident.